User Manual of the *C-Product Toolbox* *

Pablo Soto-Quiros[†], Samuel Valverde-Sanchez[‡] and Luis Chavarria-Zamora[✠]

[†,‡] *Escuela de Matemática, Instituto Tecnológico de Costa Rica, Cartago 30101, Costa Rica*
[†,✠] *Escuela de Ingeniería en Computadores, Instituto Tecnológico de Costa Rica, Cartago 30101, Costa Rica*

Emails: [†]jusoto@tec.ac.cr, [‡]savalverde@tec.ac.cr, [✠]lachavarria@tec.ac.cr
ORCID: [†]0000-0003-2903-3116, [‡]0009-0004-3023-8455, [✠]0000-0002-2510-5680

Abstract

The *C-Product Toolbox* is a computational package designed to operate with third-order tensors using the reduced c-product, an optimized version of the c-product based on the discrete cosine transform. Developed in MATLAB and PYTHON, this software improves computational efficiency by reducing memory usage and execution time, as it exclusively uses real arithmetic and avoids the complex calculations inherent to the t-product, which is based on the discrete Fourier transform. Furthermore, it incorporates advanced functionalities not available in previous tools, such as the tensor versions of the pseudoinverse and the Drazin inverse.

Contents

---

1. Introduction

1.1. General description

The *C-Product Toolbox* is a computational package designed to perform operations on third-order tensors using a variant of the c-product [1] called the *reduced c-product* [2]. This toolbox is available for both MATLAB and PYTHON, facilitating its adoption by a wide community of researchers and professionals in scientific computing and signal processing.

The reduced c-product is based on the discrete cosine transform (DCT), which improves the computational efficiency of tensor operations compared to other approaches, such as the t-product [3], which uses the discrete Fourier transform (DFT) and requires complex arithmetic. A detailed explanation of the reduced c-product is presented in Section 1.4.

1.2. Advantages of the *C-Product Toolbox*

The *C-Product Toolbox* offers advanced features not available in other similar tools, including tensor pseudoinverse, tensor Drazin inverse, and tensor equation solving by least squares. All functions have been implemented in both MATLAB and PYTHON using the NUMPY and SCIPY libraries. Being implemented in PYTHON, it allows integration into open-source environments without relying exclusively on MATLAB. This toolbox is an optimized alternative to the *Tensor-Tensor Product Toolbox 2.0* [4], based on the t-product, which has higher computational resource consumption and lacks recent updates.

*Version 1.0 - February 2025

Thanks to its formulation based on the DCT, the *C-Product Toolbox* offers key advantages, such as **lower memory consumption and reduced execution time**. Additionally, the numerical experiments reported in [2] demonstrate that this toolbox significantly improves performance in terms of computational efficiency and memory usage. Its applicability has been validated in various signal and image processing tasks, with notable results in noise removal from grayscale videos.

1.3. GitHub repository of the *C-Product Toolbox*

The source code of the *C-Product Toolbox* is available at the following GitHub repository:

`https://github.com/jusotoTEC/c-product-toolbox`

Observación 1. *The GitHub repository of the C-Product Toolbox includes all the examples and numerical experiments presented in [2].*

1.4. Formal definition of the reduced c-product

In this section, we introduce some fundamental concepts and definitions related to the *reduced c-product*. A third-order tensor $\mathcal{A} \in \mathbb{R}^{m\times n\times p}$ is a three-dimensional structure that generalizes the notion of a matrix, where each element is given by $\mathcal{A}(i,j,k) \in \mathbb{R}$, for all $i = 1,\dots,m$, $j = 1,\dots,n$, and $k = 1,\dots,p$.

Definición 1 ([1]). *Let $\mathcal{A} \in \mathbb{R}^{m\times n\times p}$ and $\mathcal{B} \in \mathbb{R}^{n\times s\times p}$. We define the **face product** as the tensor $\mathcal{C} \in \mathbb{R}^{m\times s\times p}$ given by:*

$$\mathcal{C}^{(i)} = (\mathcal{A} \triangle \mathcal{B})^{(i)} = \mathcal{A}^{(i)}\mathcal{B}^{(i)},$$

where $\mathcal{A}^{(i)}$ and $\mathcal{B}^{(i)}$ represent the i-th frontal slices of the tensors $\mathcal{A}$ and $\mathcal{B}$, respectively, for all $i = 1,\dots,p$.

Definición 2 ([1], [5]). *The **mode-3 product** between a tensor $\mathcal{A} \in \mathbb{R}^{m\times n\times p}$ and a matrix $M \in \mathbb{R}^{q\times p}$ is the tensor $\mathcal{B} \in \mathbb{R}^{m\times n\times q}$ defined as:*

$$\mathcal{B} = \mathcal{A} \times_3 M,$$

where each element of $\mathcal{B}$ is obtained through the transformation:

$$\mathcal{B}(i,j,:) = vec^{-1}(M\,vec(\mathcal{A}(i,j,:))),$$

for all $i = 1,\dots,m$ and $j = 1,\dots,n$. Here, $vec : \mathbb{R}^{1\times 1\times p} \to \mathbb{R}^p$ is the operator that converts a tubular vector in the set $\mathbb{R}^{1\times 1\times p}$ into a column vector in the set $\mathbb{R}^p$, and $vec^{-1} : \mathbb{R}^p \to \mathbb{R}^{1\times 1\times p}$ is its inverse.

Definición 3 ([1]). *The **type-II DCT matrix** is defined as:*

$$C_p(i,j) = \sqrt{\frac{2}{p(1+\delta(i,1))}}\cos\left(\frac{\pi(2j-1)(i-1)}{2p}\right),$$

for $i,j = 1,2,\dots,p$, where $\delta(i,1)$ is the Kronecker delta function, given by:

$$\delta(i,j) = \begin{cases} 1, & \text{if } i = j, \\ 0, & \text{if } i \neq j. \end{cases}$$

With these tools established, we now present the definition of the **reduced c-product**.

Definición 4 ([2]). *The **reduced c-product** between the tensors $\mathcal{A} \in \mathbb{R}^{m\times n\times p}$ and $\mathcal{B} \in \mathbb{R}^{n\times s\times p}$ is defined as:*

$$\mathcal{A} *_c \mathcal{B} = L^{-1}(L(\mathcal{A}) \triangle L(\mathcal{B})),$$

where the transformations L and L^{-1} are defined by $L(\mathcal{A}) = \mathcal{A} \times_3 C_p$ and $L^{-1}(\mathcal{A}) = \mathcal{A} \times_3 C_p^T$.

2. Description and use of the functions in the *C-Product Toolbox*

Table 1 presents a list of the functions implemented in the *C-Product Toolbox*, along with a brief description, their syntax, and the section where their use is detailed.

Function	Description	Syntax	Reference
`cprod`	Reduced c-product	`C = cprod(A,B)`	Section 2.1
`cinprod`	Tensor inner product	`c = cinprod(A,B)`	Section 2.2
`ceye`	Identity tensor	`I = ceye(n,p)`	Section 2.3
`ctransp`	Tensor transpose	`B = ctransp(A)`	Section 2.4
`csvd`	c-SVD	`[U,S,V] = csvd(A,opt)`	Section 2.5
`cqr`	c-QR	`[Q,R] = cqr(A,opt)`	Section 2.6
`csqrtt`	Tensor square root	`X = csqrtt(A)`	Section 2.7
`ctubalrank`	Tubal rank	`trank = ctubalrank(A)`	Section 2.8
`cmultirank`	Multi-rank	`mrank = cmultirank(A)`	Section 2.9
`cinv`	c-inverse	`X = cinv(A)`	Section 2.10
`cpinv`	c-pseudoinverse	`X = cpinv(A)`	Section 2.11
`cdrazin`	c-Drazin inverse and multi-index	`[X,t] = cdrazin(A)`	Section 2.12
`cnorm`	Tensor norm	`z = cnorm(A,opt)`	Section 2.13
`clowrank`	Low tubal-rank tensor approximation problem	`Ar = clowrank(A,r)`	Section 2.14
`csvt`	Tensor singular value thresholding problem	`D = csvt(A,t)`	Section 2.15
`clsq`	Tensor least squares problem	`X = tlsq(A,B,C)`	Section 2.16
`test_toolbox`	Numerical examples	`test_toolbox()`	Section 2.17

Table 1. Functions available in the *C-Product Toolbox*.

Observación 2. *For a detailed explanation of the mathematical foundations of each function in the C-Product Toolbox included in Table 1, it is recommended to consult Section 2 of [2].*

Below, the help documentation corresponding to each function in the *C-Product Toolbox* is presented.

2.1. `cprod`

```
    Description: Computes the reduced c-product between two tensors
Function syntax: C = cprod(A, B)
          Input: A = tensor of dimension  m1 x n1 x p1
                 B = tensor of dimension  m2 x n2 x p2
         Output: C = tensor of dimension  m1 x n2 x p1
```

2.2. cinprod

```
    Description: Calculates the inner product between two tensors
Function syntax: c = cinprod(A,B)
          Input: A = tensor of dimension m x n x p
                 B = tensor of dimension n x m x p
         Output: c = inner product between tensors A and B
```

2.3. ceye

```
    Description: Computes the identity tensor under the reduced c-product
Function syntax: I = ceye(n,p)
          Input: n = positive integer
                 p = positive integer
         Output: I = tensor of dimension n x n x p
```

2.4. ctransp

```
    Description: Computes the transpose of a tensor under the
                 reduced c-product
Function syntax: B = ctransp(A)
          Input: A = tensor of dimension m x n x p
         Output: B = tensor of dimension n x m x p
```

2.5. csvd

```
    Description: Computes the SVD decomposition of a tensor
                 under the reduced c-product.
Function syntax: [U, S, V] = cpinv(A, opt)
          Input: A = tensor of size m x n x p
               opt = options for various results of U, S, and V
                     1) 'full': (default) produces the full SVD decomposition
                                of the tensor, i.e., A = U*S*V^T, where:
                                U  = tensor of size m x m x p
                                S  = tensor of size m x n x p
                                V  = tensor of size n x n x p
                     2) 'econ': produces the 'economy-sized' decomposition.
                                Let s = min(m,n). Then, A = U*S*V^T, where:
                                U = tensor of size m x s x p
                                S = tensor of size s x s x p
                                V = tensor of size n x s x p
         Output: U, S, V
```

2.6. cqr

```
    Description: Computes the tensor QR decomposition
                 under the reduced c-product
Function Syntax: [Q,R] = cqr(A,opt)
          Input: A = tensor of dimension m x n x p
               opt = options for various Q and R results
                     1) 'full': (default) produces the full
                                tensor QR decomposition, i.e., A = Q*R,
                                where:
                                Q  = tensor of dimension m x m x p
                                R  = tensor of dimension m x n x p
                     2) 'econ': produces the 'economy-sized' decomposition
                                when m > n. Let A = Q*R, where:
                                Q = tensor of dimension m x n x p
                                R = tensor of dimension n x n x p
                                If m <= n, then the 'economy-sized'
                                decomposition is the same as the
                                full decomposition
         Output: Q, R
```

2.7. csqrtt

```
    Description: Computes the principal square root of a tensor
                 under the reduced c-product
Function syntax: X = csqrtt(A)
          Input: A = tensor of dimension m x m x p
         Output: X = tensor of dimension m x m x p
```

2.8. ctubalrank

```
    Description: Computes the tubal rank of a tensor using
                 the c-SVD method
Function syntax: trank = ctubalrank(A)
         Inputs: A = tensor of dimension m x n x p
         Output: trank = non-negative integer
```

2.9. cmultirank

```
    Description: Computes the multi-rank of a tensor using
                 the reduced c-product
Function syntax: mrank = cmultirank(A)
         Inputs: A = tensor of dimension m x n x p
         Output: mrank = vector of dimension p
```

2.10. cinv

```
    Description: Computes the tensor inverse
                 under the reduced c-product
Function syntax: X = cinv(A)
         Inputs: A = tensor of dimension m x m x p
         Output: X = tensor of dimension m x m x p
```

2.11. cpinv

```
    Description: Computes the tensor pseudoinverse
                 under the reduced c-product
Function syntax: X = cpinv(A)
         Inputs: A = tensor of dimension m x n x p
         Output: X = tensor of dimension n x m x p
```

2.12. cdrazin

```
    Description: Computes the Drazin inverse of a tensor and
                 the multi-index of a tensor under the c-product
Function syntax: [X,t] = cdrazin(A)
         Inputs: A = tensor of dimension m x m x p
        Outputs: X = tensor of dimension m x m x p
                 t = vector of dimension p
```

2.13. cnorm

```
    Description: Computes the tensor norm under the c-product
Function syntax: z = cnorm(A,opt)
         Inputs: A = tensor of dimension m x n x p
               opt = options for different types of tensor norms:
                     1) 'fro' : Frobenius norm
                     2) 'spec': Spectral norm
                     3) 'nuc' : Nuclear norm
         Output: z = non-negative number
```

2.14. clowrank

```
    Description: Computes the solution to the low tubal-rank
                 tensor problem using the reduced c-product
Function syntax: Ar = clowrank(A,r)
         Inputs: A  = tensor of dimension m x n x p
                 r  = positive integer such that r <= min(m,n)
         Output: Ar = tensor of dimension m x n x p
```

2.15. csvt

```
    Description: Computes the solution to the tensor singular value
                 thresholding problem under the reduced c-product
Function syntax: X = csvt(A,t)
         Inputs: A = tensor of dimension m x n x p
                 t = positive constant
         Output: X = tensor of dimension m x n x p
```

2.16. clsq

```
    Description: Computes the solution to the tensor least squares
                 problem under the reduced c-product
Function syntax: X = clsq(A,B,C)
         Inputs: A = tensor of dimension  m x n x p
                 B = tensor of dimension  m x r x p
                 C = tensor of dimension  s x n x p
         Output: X = tensor of dimension  r x s x p
```

2.17. test_toolbox

```
    Description: Numerical examples to evaluate the performance and
                 functionality of the C-Product Toolbox
Function syntax: test_toolbox()
```

3. Quick guide to install the *C-Product Toolbox* in Matlab

This manual explains how to download and install the `cproduct` package in MATLAB from GitHub.

3.1. Download the `cproduct.zip` File

1. Visit the following link in your browser:

   https://github.com/jusotoTEC/c-product-toolbox/tree/main/cproduct_matlab

2. Download the `cproduct.zip` file to your computer.
3. Extract the contents of the file. This will create a folder named `cproduct`.

3.2. Add the toolbox to the Matlab path

To make MATLAB recognize the *C-Product Toolbox*, follow these steps:

1. Open MATLAB.
2. In the command window, run the following command:

```
addpath('path/to/cproduct');
savepath;
```

   Replace `'path/to/cproduct'` with the actual location of the folder `cproduct` on your system.

3.3. Verify the installation

To check that the functions are available, run the following in the command line:

```
which function_name
```

For example:

```
which cprod
```

If the installation was successful, MATLAB will display the location of the corresponding file.

3.4. Using the *C-Product Toolbox* in Matlab

You can now invoke the functions from the *C-Product Toolbox* just like any other MATLAB function. Figure 1 shows examples of how to use all the available functions from the *C-Product Toolbox* within the MATLAB environment.

```
% Example of applying the C-Product Toolbox functions in MATLAB

A = rand(10,5,3);          % Third-order tensor generated ...
                           % ... randomly with uniform distribution
B = rand(5,5,3);           % Third-order tensor generated ...
                           % ... randomly with uniform distribution
C = cprod(A,B);            % c-product between A and B
c = cprod(A,B);            % Tensorial inner product between A and B
I = ceye(10,5);            % Identity tensor of size 10 x 10 x 5
At = ctransp(A);           % Tensorial transpose of A
[U,S,V] = csvd(A);         % c-SVD of A
[Q,R] = cqr(A);            % c-QR of A
X1 = cqr(A);               % Tensorial square root of A
trank = ctubalrank(A);     % Tubal rank of A
mrank = cmultirank(A);     % Multi-rank of A
X2 = cinv(B);              % c-inverse of B
X3 = cpinv(A);             % c-pseudoinverse of A
[X4,t] = cdrazin(B);       % c-Drazin inverse of B and multi-index
z1 = cnorm(A,'fro');       % Frobenius tensor norm of A
z2 = cnorm(A,'spec');      % Spectral tensor norm of A
z3 = cnorm(A,'nuc');       % Nuclear tensor norm of A
Ar = clowrank(A,3);        % Tensor solution for the low-tubal-rank ...
                           % ... approximation problem
X5 = csvt(A,0.5);          % Tensor solution for the tensor singular value ...
                           % ... thresholding problem
A1 = rand(5,4,3);          % Third-order tensor generated ...
                           % ... randomly with uniform distribution
B1 = rand(5,2,3);          % Third-order tensor generated ...
                           % ... randomly with uniform distribution
C1 = rand(2,4,3);          % Third-order tensor generated ...
                           % ... randomly with uniform distribution
X6 = clsq(A1,B1,C1);       % Tensor solution for the tensor least squares problem
```

Figure 1. Numerical Example of the *C-Product Toolbox* in MATLAB

3.5. (Optional) Automatically load the *C-Product Toolbox* at Matlab startup

If you want the toolbox to load automatically each time you start MATLAB, add the line `addpath('path/to/cproduct');` to the `startup.m` file. To do so:

1. In MATLAB, run the command:

```
edit startup
```

2. Add the line:

```
addpath('path/to/cproduct');
```

3. Save and close the file.

Following these steps, the *C-Product Toolbox* will be ready to use in MATLAB.

4. Quick guide to install the *C-Product Toolbox* in Python

This manual explains how to download and install the `cproduct` package in PYTHON from GitHub.

4.1. Download the `cproduct.zip` File

1. Visit the following link in your browser:

   https://github.com/jusotoTEC/c-product-toolbox/tree/main/cproduct_python

2. Download the `cproduct.zip` file to your computer.
3. Extract the contents of the file. This will create a folder named `cproduct`.

4.2. Package Installation

1. Open a terminal or command prompt.
2. Navigate into the extracted `cproduct` folder:

```
cd path/to/extracted/cproduct
```

3. Run the following command to install the package in editable mode:

```
pip install .
```

4.3. Verify the Installation

After installation, open Python and run:

```
import cproduct as cp
print(cp.__name__)  # Should print "cproduct"
```

4.4. Using the *C-Product Toolbox* in Python

To use the functions of the *C-Product Toolbox*, import the module into your script:

```
import cproduct as cp
```

Figure 2 shows the usage of all the functions of the *C-Product Toolbox* in PYTHON.

```python
# Example application of the C-Product Toolbox functions in Python

import cproduct as cp          # Import the C-Product Toolbox
import numpy as np             # Import NumPy

A = np.random.rand(10,5,3)     # Third-order tensor generated ...
                               # ... randomly with uniform distribution
B = np.random.rand(5,5,3)      # Third-order tensor generated ...
                               # ... randomly with uniform distribution
C = cp.cprod(A,B);             # c-product between A and B
c = cp.cprod(A,B)              # Tensor inner product between A and B
I = cp.ceye(10,5)              # Identity tensor of size 10 x 10 x 5
At = cp.ctransp(A)             # Tensor transpose of A
[U,S,V] = cp.csvd(A)           # c-SVD of A
[Q,R] = cp.cqr(A)              # c-QR of A
X1 = cp.cqr(A)                 # Tensor square root of A
trank = cp.ctubalrank(A)       # Tubal rank of A
mrank = cp.cmultirank(A)       # Multi-rank of A
X2 = cp.cinv(B)                # c-inverse of B
X3 = cp.cpinv(A)               # c-pseudoinverse of A
[X4,t] = cp.cdrazin(B)         # c-Drazin inverse of B and multi-index
z1 = cp.cnorm(A,'fro')         # Frobenius tensor norm of A
z2 = cp.cnorm(A,'spec')        # Spectral tensor norm of A
z3 = cp.cnorm(A,'nuc')         # Nuclear tensor norm of A
Ar = cp.clowrank(A,3)          # Tensor solution for the low-tubal-rank
                               # ... approximation problem
X5 = cp.csvt(A,0.5)            # Tensor solution of the tensor singular ...
                               # ...  value thresholding problem
A1 = np.random.rand(5,4,3)     # Third-order tensor generated ...
                               # ... randomly with uniform distribution
B1 = np.random.rand(5,2,3)     # Third-order tensor generated ...
                               # ... randomly with uniform distribution
C1 = np.random.rand(2,4,3)     # Third-order tensor generated ...
                               # ... randomly with uniform distribution
X6 = cp.clsq(A1,B1,C1)         # Tensor solution of the tensor least ...
                               # ... squares problem
```

Figure 2. Numerical example of the *C-Product Toolbox* in PYTHON

4.5. Uninstalling the Package

If you wish to remove the package, use:

```
pip uninstall cproduct
```

References

[1] E. Kernfeld, M. Kilmer, and S. Aeron, "Tensor–tensor products with invertible linear transforms", *Linear Algebra and its Applications*, vol. 485, pp. 545–570, 2015.

[2] P. Soto-Quiros, "*C-product toolbox*: A computational package for third-order tensor operations based on the reduced *c*-product", *Journal of Computational Mathematics and Data Science*, 2025, Under review.

[3] M. E. Kilmer and C. D. Martin, "Factorization strategies for third-order tensors", *Linear Algebra and its Applications*, vol. 435, no. 3, pp. 641–658, 2011.

[4] C. Lu, *Tensor-tensor product toolbox*, `https://github.com/canyilu/tproduct`, Carnegie Mellon University, Jun. 2018.

[5] H. Jin, S. Xu, Y. Wang, and X. Liu, "The Moore–Penrose inverse of tensors via the m-product", *Computational and Applied Mathematics*, vol. 42, no. 6, p. 294, 2023.